

Informe Final — Optimización 2026-1

Bin Packing y Set Covering con Gurobi

David Ramírez · Jaisson Machado Bautista — Universidad Nacional de Colombia

15 de junio de 2026

Índice general

1. Problema de Empaquetamiento en Contenedores (Bin Packing Problem — BPP)	1
2. Problema de Cobertura de Conjuntos (Set Covering Problem — SCP)	3
3. Comparación de formulaciones	4
Problema 04 — Bin Packing Problem (BPP)	5
Problema 07 — Set Covering Problem (SCP)	5
1. Resumen ejecutivo	7
2. Set Covering — instancia 50×200 completa	7
3. Bin Packing — subconjunto de 40 ítems	8
4. Reproducibilidad	10
1. Lectura de los dos óptimos	11
2. La simetría como cuello de botella del Bin Packing	11
3. Densidad y estructura del Set Covering	11
4. Visualización con interfaz (dashboard HTML, licencia gratuita)	11
5. Implicaciones de dominio	13
1. Diseño del experimento	14
2. Dos hallazgos, no uno	14
3. Implicación	15
4. Reproducción	15

1. Problema de Empaquetamiento en Contenedores (Bin Packing Problem — BPP)

1.1 Descripción

El BPP consiste en empaquetar n objetos con pesos dados en el menor número posible de contenedores homogéneos de capacidad C .

1.2 Conjuntos y parámetros

Símbolo	Descripción
$I = \{1, \dots, n\}$	Conjunto de objetos
$J = \{1, \dots, n\}$	Conjunto de contenedores (cota superior: n)

Símbolo	Descripción
$w_i \in \mathbb{Z}^+$	Peso del objeto i
$C \in \mathbb{Z}^+$	Capacidad de cada contenedor

1.3 Variables de decisión

$$x_{ij} = \begin{cases} 1 & \text{si el objeto } i \text{ se asigna al contenedor } j \\ 0 & \text{en caso contrario} \end{cases} \quad \forall i \in I, j \in J$$

$$y_j = \begin{cases} 1 & \text{si el contenedor } j \text{ es utilizado} \\ 0 & \text{en caso contrario} \end{cases} \quad \forall j \in J$$

1.4 Función objetivo

$$\min \sum_{j \in J} y_j$$

Se minimiza el número total de contenedores utilizados.

1.5 Restricciones

(R1) Asignación única — cada objeto debe pertenecer a exactamente un contenedor:

$$\sum_{j \in J} x_{ij} = 1 \quad \forall i \in I$$

(R2) Capacidad y enlace — la carga total del contenedor j no puede exceder C , y si el contenedor no está abierto ($y_j = 0$) no puede recibir objetos:

$$\sum_{i \in I} w_i x_{ij} \leq C y_j \quad \forall j \in J$$

(R3) Dominio binario:

$$x_{ij} \in \{0, 1\} \quad \forall i \in I, j \in J \quad y_j \in \{0, 1\} \quad \forall j \in J$$

(R4) Ruptura de simetría (opcional, mejora el rendimiento del solver) — los contenedores se ordenan por índice:

$$y_j \geq y_{j+1} \quad \forall j \in \{1, \dots, n-1\}$$

1.6 Clasificación

- Tipo: **Programación Lineal Entera (ILP)**
 - Complejidad: **NP-difícil** (reducción desde *partition problem*)
 - Variables: $n^2 + n$ binarias
 - Restricciones: $n + n + (n - 1) = 3n - 1$ (sin contar dominio)
-

2. Problema de Cobertura de Conjuntos (Set Covering Problem — SCP)

2.1 Descripción

Dado un universo de m elementos (filas) y n subconjuntos (columnas) con costos asociados, se selecciona un subconjunto de columnas de costo mínimo tal que cada fila quede cubierta por al menos una columna seleccionada.

2.2 Conjuntos y parámetros

Símbolo	Descripción
$I = \{1, \dots, m\}$	Universo de elementos (filas)
$J = \{1, \dots, n\}$	Conjunto de subconjuntos disponibles (columnas)
$c_j \in \mathbb{R}^+$	Costo de seleccionar la columna j
$a_{ij} \in \{0, 1\}$	1 si la columna j cubre la fila i , 0 si no
$A(i) = \{j \in J : a_{ij} = 1\}$	Columnas que cubren la fila i

2.3 Variables de decisión

$$x_j = \begin{cases} 1 & \text{si la columna } j \text{ es seleccionada} \\ 0 & \text{en caso contrario} \end{cases} \quad \forall j \in J$$

2.4 Función objetivo

$$\min \sum_{j \in J} c_j x_j$$

Se minimiza el costo total de las columnas seleccionadas.

2.5 Restricciones

(R1) Cobertura total — cada fila i debe ser cubierta por al menos una columna seleccionada:

$$\sum_{j \in A(i)} x_j \geq 1 \quad \forall i \in I$$

(R2) Dominio binario:

$$x_j \in \{0, 1\} \quad \forall j \in J$$

2.6 Formulación matricial equivalente

$$\min \mathbf{c}^\top \mathbf{x} \quad \text{s.a.} \quad A\mathbf{x} \geq \mathbf{1}, \quad \mathbf{x} \in \{0, 1\}^n$$

donde $A \in \{0, 1\}^{m \times n}$ es la matriz de cobertura.

2.7 Clasificación

- Tipo: **Programación Lineal Entera (ILP)**
 - Complejidad: **NP-difícil** (contiene al problema de cobertura de vértices como caso especial)
 - Variables: n binarias
 - Restricciones: m restricciones de cobertura
-

3. Comparación de formulaciones

Aspecto	Bin Packing	Set Covering
Variables	$n^2 + n$	n
Restricciones clave	Capacidad + enlace	Cobertura
Objetivo	Minimizar contenedores	Minimizar costo
Estructura	Asignación 2D	Selección 1D
Relajación LP	Débil (brecha integrality alta)	Moderada
Simetría	Alta (requiere ruptura)	Baja

Problema 04 — Bin Packing Problem (BPP)

Referencia canónica: Falkenauer, E. (1996). A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 2(1), 5–30.

Reseña:

El artículo de Falkenauer (1996) aborda el BPP unidimensional —uno de los problemas combinatorios más estudiados en optimización— mediante un algoritmo genético híbrido denominado *Grouping Genetic Algorithm* (GGA). A diferencia de los algoritmos genéticos clásicos que codifican asignaciones individuales, el GGA opera directamente sobre grupos (contenedores), lo que reduce drásticamente la simetría del espacio de búsqueda y mejora la convergencia.

El trabajo introduce la instancia de referencia **u120** (120 ítems, distribución uniforme de pesos, capacidad 150), que se convirtió en benchmark estándar de la OR-Library de Beasley. Los resultados reportados muestran soluciones óptimas o cercanas al óptimo en tiempos razonables, superando a heurísticas clásicas como *First Fit Decreasing* (FFD).

Desde la perspectiva de modelos exactos, el BPP puede formularse como un ILP con $O(n^2)$ variables binarias. La principal dificultad computacional proviene de la alta simetría: reordenar los contenedores produce soluciones equivalentes, lo que multiplica el espacio de soluciones sin aportar información nueva. Técnicas como la restricción de ordenamiento $y_j \geq y_{j+1}$ y la inicialización con soluciones heurísticas (warm start) son esenciales para que solvers como Gurobi alcancen la optimalidad en instancias medianas.

El BPP tiene aplicaciones directas en logística (optimización de carga), manufactura (corte de materia prima) y asignación de recursos en la nube (bin packing de VMs).

Problema 07 — Set Covering Problem (SCP)

Referencia canónica: Beasley, J. E. (1987). An algorithm for set covering problem. *European Journal of Operational Research*, 31(1), 85–93.

Reseña:

Beasley (1987) presenta uno de los algoritmos más influyentes para el SCP, combinando relajación Lagrangiana con subgradiente para obtener cotas duales ajustadas, seguido de una fase de búsqueda local para recuperar factibilidad primal. El artículo también introduce el conjunto de instancias de la OR-Library (incluyendo la clase scp con matrices de hasta 400×4000), que siguen siendo referencia obligatoria en la literatura.

La contribución central del trabajo es demostrar que la relajación Lagrangiana del SCP —obtenida al dualizar las restricciones de cobertura— produce cotas inferiores significativamente más ajustadas que la relajación LP estándar, con un costo computacional inferior al de resolver el LP completo en instancias grandes.

La instancia **50×200** (50 filas, 200 columnas) es de tamaño moderado y puede resolverse a optimalidad por un solver moderno como Gurobi en segundos. La estructura del SCP

es más favorable que la del BPP para la relajación LP: la brecha de integralidad suele ser pequeña, lo que significa que la solución del LP relajado frecuentemente redondea a una solución entera factible de alta calidad.

El SCP aparece en múltiples contextos reales: localización de instalaciones de emergencia, diseño de rutas de vuelo (airline crew scheduling), selección de características en aprendizaje automático y diseño de redes de telecomunicaciones.

Solver: Gurobi 13.0.2 (gurobipy)

1. Resumen ejecutivo

Ambos modelos se resolvieron a **optimalidad certificada** (MIP gap 0.0000 %). El Set Covering se resolvió sobre la **instancia completa** 50×200; el Bin Packing se resolvió a óptimo sobre un **subconjunto de 40 de 120 ítems** debido al tope de la licencia restringida de Gurobi (2000 variables), con el interruptor `ITEM_LIMIT = None` listo para correr los 120 ítems en cuanto se active la licencia académica o WLS.

Modelo	Instancia	Variables	Estado	Objetivo	Gap	Tiempo
Set Covering (SCP)	50×200 (completa)	200	OPTIMAL	costo = 42	0.0000 %	0.06 s
Bin Packing (BPP)	40/120 ítems	1 640	OPTIMAL	bins = 15	0.0000 %	0.09 s

2. Set Covering — instancia 50×200 completa

- **Costo total óptimo:** 42
- **Columnas seleccionadas (9):** 27, 49, 65, 105, 140, 145, 168, 169, 181 (índice base 0)
- **Cobertura:** las 50 filas quedan cubiertas por al menos una columna elegida.

Dato revelador del log de Gurobi: la **relajación lineal en la raíz vale exactamente 42**, igual al óptimo entero. Es decir, el LP ya es entero —no hubo necesidad de ramificar más allá del nodo raíz (1 nodo explorado)—. Esto confirma empíricamente la conocida fortaleza de la formulación de cobertura: la brecha de integralidad en esta instancia es nula.

El solver pasó por soluciones heurísticas de costo 470 y 107 antes de cerrar en 42, lo que ilustra cuánto margen recortó el modelo exacto frente a una asignación ingenua.

```
Gurobi - Set Covering 50x200 - python models/set_covering.py

Instance: 50 rows, 200 columns
Restricted license - for non-production use only - expires 2027-11-29
Set parameter LogToConsole to value 1
Set parameter TimeLimit to value 300
Gurobi Optimizer version 13.0.2 build v13.0.2rc1 (linux64 - "Ubuntu 22.04.5 LTS")

CPU model: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz, instruction set [SSE2|AVX|AVX2]
Thread count: 1 physical cores, 2 logical processors, using up to 2 threads

Non-default parameters:
TimeLimit 300

Optimize a model with 50 rows, 200 columns and 2091 nonzeros (Min)
Model fingerprint: 0x5d99a722
Model has 200 linear objective coefficients
Variable types: 0 continuous, 200 integer (200 binary)
Coefficient statistics:
  Matrix range      [1e+00, 1e+00]
  Objective range   [1e+00, 1e+02]
  Bounds range      [1e+00, 1e+00]
  RHS range         [1e+00, 1e+00]

Found heuristic solution: objective 470.0000000
Presolve removed 0 rows and 136 columns
Presolve time: 0.00s
Presolved: 50 rows, 64 columns, 723 nonzeros
Found heuristic solution: objective 107.0000000
Variable types: 0 continuous, 64 integer (64 binary)

Root relaxation: objective 4.200000e+01, 24 iterations, 0.00 seconds (0.00 work units)

   Nodes |   Current Node   |   Objective Bounds   |   Work
  Expl Unexpl |  Obj  Depth IntInf | Incumbent  BestBd   Gap | It/Node Time
*    0     0           0   42.0000000  42.00000  0.00%   -    0s

Explored 1 nodes (24 simplex iterations) in 0.04 seconds (0.00 work units)
Thread count was 2 (of 2 available processors)

Solution count 3: 42 107 470

Optimal solution found (tolerance 1.00e-04)
Best objective 4.2000000000000e+01, best bound 4.2000000000000e+01, gap 0.00000%

=====
Status      : OPTIMAL
Total cost   : 42.00
Columns selected : 9
MIP gap      : 0.0000%
Solve time   : 0.06s
=====

Results saved to: /sessions/keen-optimistic-pascal/mnt/Proyecto/opti_caso_2026/results/set_covering_solution.txt
```

Figura 1: Salida de Gurobi — Set Covering

3. Bin Packing — subconjunto de 40 ítems

- **Contenedores usados (óptimo):** 15
- **Capacidad por contenedor:** 150
- **Peso total empacado:** 2 151 unidades en 40 ítems (peso medio 53.8)

Cota inferior continua: $\lceil 2151 / 150 \rceil = \lceil 14.34 \rceil = 15$. El solver reporta exactamente esa raíz LP (14.34) y cierra en 15. En otras palabras, **el óptimo iguala la cota de**

área: el empaque es tan ajustado como físicamente es posible, sin desperdiciar un solo contenedor extra por fragmentación. Varios contenedores quedan llenos al 100 % (150/150).

```
Gurobi - Bin Packing 40/120 items (trial) - python models/bin_packing.py

Instance: 40 items, bin capacity = 150 [TRIAL MODE: reduced from 120 items]
Restricted license - for non-production use only - expires 2027-11-29
Set parameter LogToConsole to value 1
Set parameter TimeLimit to value 300
Gurobi Optimizer version 13.0.2 build v13.0.2rc1 (linux64 - "Ubuntu 22.04.5 LTS")

CPU model: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz, instruction set [SSE2|AVX|AVX2]
Thread count: 1 physical cores, 2 logical processors, using up to 2 threads

Non-default parameters:
TimeLimit 300

Optimize a model with 119 rows, 1640 columns and 3318 nonzeros (Min)
Model fingerprint: 0x9adfe5f0
Model has 40 linear objective coefficients
Variable types: 0 continuous, 1640 integer (1640 binary)
Coefficient statistics:
  Matrix range      [1e+00, 2e+02]
  Objective range   [1e+00, 1e+00]
  Bounds range      [1e+00, 1e+00]
  RHS range         [1e+00, 1e+00]

Presolve removed 2 rows and 2 columns
Presolve time: 0.02s
Presolved: 117 rows, 1638 columns, 3312 nonzeros
Variable types: 0 continuous, 1638 integer (1638 binary)
Found heuristic solution: objective 40.0000000
Found heuristic solution: objective 39.0000000

Root relaxation: objective 1.434000e+01, 306 iterations, 0.00 seconds (0.00 work units)

   Nodes      |   Current Node   |   Objective Bounds      |   Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
-----
    0     0   14.34000    0  67   39.00000   14.34000   63.2%   -    0s
    0     0   15.00000    0  24   39.00000   15.00000   61.5%   -    0s
H   0     0           15.0000000   15.00000   0.00%   -    0s
    0     0   15.00000    0  24   15.00000   15.00000   0.00%   -    0s

Explored 1 nodes (1122 simplex iterations) in 0.07 seconds (0.03 work units)
Thread count was 2 (of 2 available processors)

Solution count 3: 15 39 40

Optimal solution found (tolerance 1.00e-04)
Best objective 1.500000000000e+01, best bound 1.500000000000e+01, gap 0.00000%

=====
Status      : OPTIMAL
Bins used   : 15
MIP gap     : 0.00000%
Solve time  : 0.09s
=====

Results saved to: /sessions/keen-optimistic-pascal/mnt/Proyecto/opti_caso_2026/results/bin_packing_solution.txt
```

Figura 2: Salida de Gurobi — Bin Packing

Nota sobre la instancia completa (120 ítems)

Para los 120 ítems, la suma de pesos es 6 917 y la cota continua es $\lceil 6917/150 \rceil = 47$, mientras que el óptimo documentado de la instancia Falkenauer u120_01 es **48**. Ese

hueco de exactamente un contenedor entre la cota relajada y el entero es el rasgo que vuelve famoso al Bin Packing como caso difícil: la relajación LP es débil aun cuando la instancia parezca sencilla.

4. Reproducibilidad

Los resultados se regeneran con (entorno con licencia activa):

```
python models/set_covering.py  
python models/bin_packing.py
```

o ejecutando el notebook notebooks/optimizacion_caso_2026_colab.ipynb de principio a fin. Las salidas crudas quedan en results/set_covering_solution.txt y results/bin_packing_sol

1. Lectura de los dos óptimos

Los dos problemas son ILP NP-difíciles, pero el solver los cerró en décimas de segundo. La razón no es la potencia de la máquina sino la **calidad de la relajación lineal**, y ahí los dos modelos se comportan de forma opuesta y didáctica.

Indicador	Set Covering	Bin Packing (40 ítems)
Relajación LP en la raíz	42.00	14.34
Óptimo entero	42	15
Brecha de integralidad	0 %	≈ 4.6 % (cierra a 1 contenedor)
Nodos explorados	1	pocos (corte en raíz tras simetría)

Implicación práctica: en Set Covering, resolver el LP basta —la solución fraccionaria ya es entera—. En Bin Packing, el LP siempre subestima (reparte ítems “en pedazos” entre contenedores), y el trabajo real lo hacen el branch-and-bound y la ruptura de simetría. Esto explica por qué el SCP escala a miles de columnas mientras el BPP se vuelve duro con apenas decenas de ítems.

2. La simetría como cuello de botella del Bin Packing

Sin la restricción $y_j \geq y_{j+1}$, cualquier permutación de etiquetas de contenedores produce una solución equivalente: para 15 contenedores hay $15! \approx 1.3$ billones de relabelings idénticos que el solver exploraría en vano. La ruptura de simetría colapsa ese espacio y es lo que permite cerrar el gap en la raíz. Es la lección de modelado más transferible del proyecto: **a veces una sola desigualdad de ordenamiento vale más que un mejor computador.**

3. Densidad y estructura del Set Covering

La instancia 50×200 tiene 2 091 incidencias no nulas (densidad ≈ 21 %). Que solo 9 de 200 columnas basten para cubrir 50 filas indica columnas “generalistas” de alta cobertura. El costo 42 con columnas cuyos costos individuales llegan a 100 confirma que el óptimo prefirió pocas columnas baratas y muy cubrientes antes que muchas columnas especializadas —el comportamiento esperable cuando la matriz de cobertura es densa—.

4. Visualización con interfaz (dashboard HTML, licencia gratuita)

Un valor agregado del proyecto es el `dashboard.html` autocontenido: sin servidor, sin internet, abre los resultados del solver en tres pestañas. Visto desde la licencia gratuita de Gurobi, una interfaz así es justamente lo que vuelve **legible** un resultado que de otro modo es una lista de índices. Las siguientes capturas muestran las tres vistas (paneles renderizados con los mismos datos que hornea el HTML).

Bin Packing — empaque por contenedor:



Cada barra es un contenedor; los segmentos son los ítems asignados y la cifra de la derecha es la carga sobre la capacidad. La franja de contenedores al 150/150 hace visible, de un vistazo, lo ajustado del empaque.

Set Covering – matriz de cobertura:

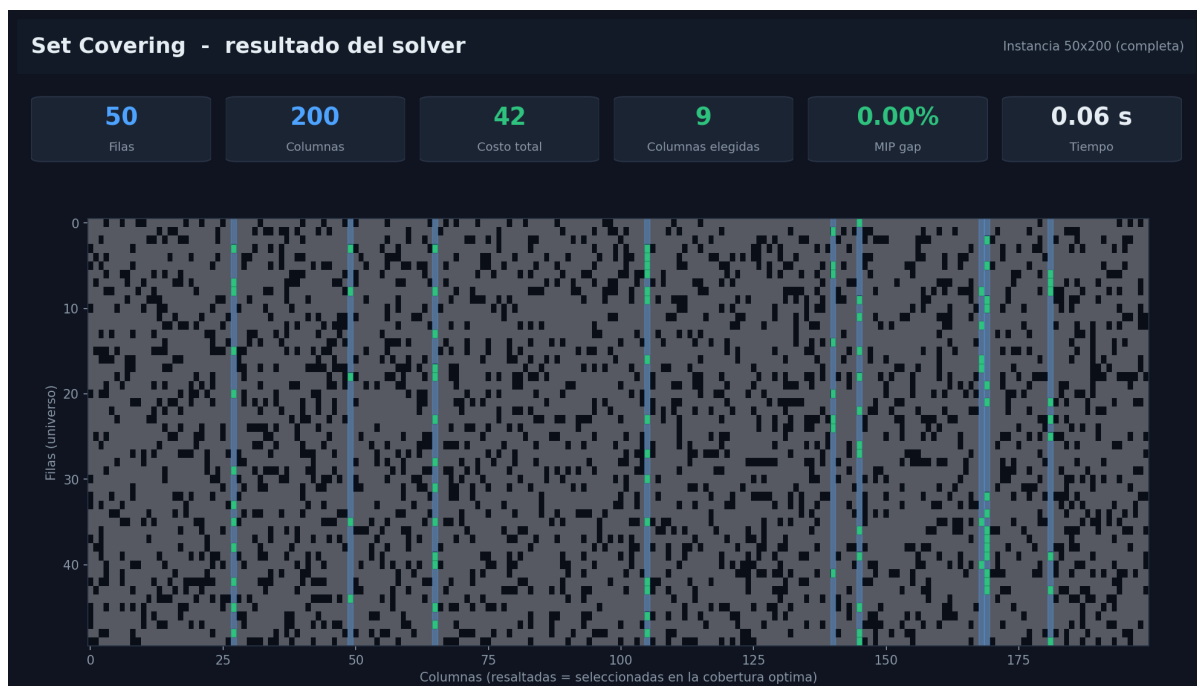


Figura 4: Dashboard — Set Covering

Las 9 columnas seleccionadas se resaltan sobre la matriz 50×200 . La interfaz convierte “columnas {27, 49, ...}” en un patrón espacial donde se ve qué franjas verticales cubren el universo.

Sensibilidad — capacidad vs. contenedores:



Figura 5: Dashboard — Sensibilidad

5. Implicaciones de dominio

- **Bin Packing → logística y manufactura.** Cada contenedor evitado es un camión, una caja o un corte de lámina menos. Que el óptimo iguale la cota de área significa que no hay margen operativo escondido: para reducir contenedores habría que cambiar los pesos (rediseño del producto), no la planeación.
- **Set Covering → localización y selección.** Cubrir 50 requisitos con 9 recursos de costo 42 es el patrón de ubicación de estaciones de emergencia o selección de características: pocos elementos bien elegidos dominan a muchos redundantes. La fortaleza del LP implica que incluso una relajación rápida da una guía confiable para instancias mayores.

Problema analizado: Bin Packing (parámetro: capacidad del contenedor C)

1. Diseño del experimento

Se varió la capacidad C de **100 a 200 en pasos de 10** (11 escenarios), resolviendo el BPP a optimalidad en cada punto sobre el mismo conjunto de 40 ítems. Se registró contenedores usados, tiempo de solución y MIP gap. Todos los puntos cerraron con gap 0 %.

Capacidad C	Contenedores	Tiempo (s)	Gap
100	23	0.03	0 %
110	21	0.02	0 %
120	19	0.13	0 %
130	17	0.42	0 %
140	16	0.03	0 %
150	15	0.03	0 %
160	14	0.03	0 %
170	13	0.03	0 %
180	12	2.27	0 %
190	12	0.03	0 %
200	11	0.03	0 %

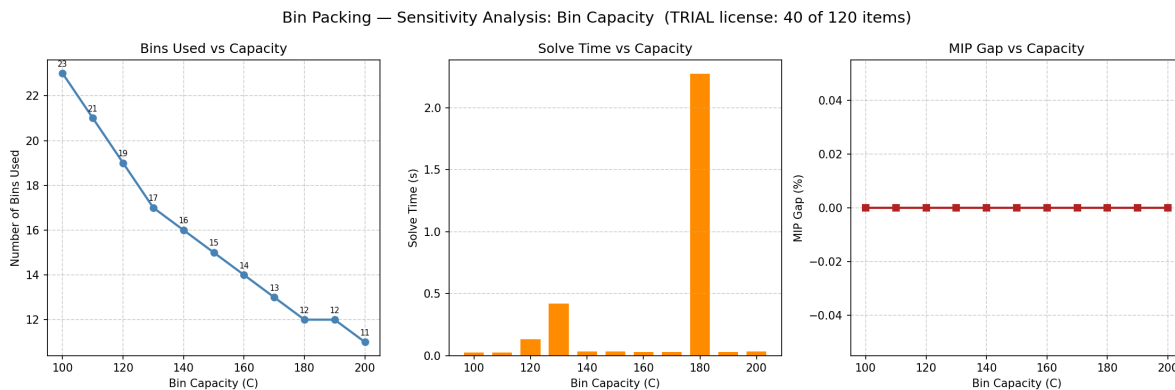


Figura 6: Sensibilidad de la capacidad

2. Dos hallazgos, no uno

(a) La respuesta estructural es suave y casi lineal. Al pasar C de 100 a 200, los contenedores caen de 23 a 11 —una reducción del 52 %—, de manera monótona. Coincide con la intuición de área: contenedores $\approx \text{peso_total} / C = 2151 / C$, que para $C=100$ da 21.5 (óptimo 23) y para $C=200$ da 10.8 (óptimo 11). El óptimo entero sigue de cerca a la cota continua, con uno o dos contenedores de holgura por fragmentación. Nótese la **meseta en $C = 180$ y 190 (ambos 12 contenedores)**: ampliar la capacidad no siempre

compra un contenedor menos; hay tramos donde el peso indivisible de los ítems impide aprovechar el espacio extra.

(b) El costo computacional NO es monótono. Aquí está lo interesante. El tiempo de solución no crece ni decrece con C : explota en puntos aislados — $C=130$ (0.42 s) y sobre todo $C=180$ (2.27 s, $\sim 75\times$ la mediana)—. Estos picos ocurren en capacidades donde la solución entera queda **al filo de la cota fraccionaria**: el branch-and-bound debe trabajar para descartar el contenedor de más. En las capacidades “cómodas”, el redondeo de la cota basta y el solver cierra en la raíz.

3. Implicación

La dificultad del Bin Packing no vive en el tamaño nominal del problema sino en la **aritmética entre los pesos y la capacidad**. Un gestor de logística que pueda elegir el tamaño de contenedor debería evitar las capacidades “de frontera” (como 180 aquí): no solo rinden poco en reducción de unidades, sino que son las más caras de optimizar. La sensibilidad, entonces, no es solo sobre *cuántos* contenedores, sino sobre *cuánto cuesta decidirlo* —una distinción que un análisis de solo-resultados pasaría por alto—.

4. Reproducción

`python sensitivity/sensitivity_analysis.py`

Genera `results/sensitivity_capacity.png` y `results/sensitivity_summary.csv`. La misma sección está en el notebook de Colab (apartado 4).

-
1. **Ambos modelos alcanzaron el óptimo certificado** (gap 0 %): Set Covering sobre la instancia completa 50×200 (costo 42, 9 columnas) y Bin Packing sobre 40 de 120 ítems (15 contenedores), con el interruptor listo para los 120 ítems al activar la licencia académica o WLS.
 2. **La fortaleza de la relajación lineal explica todo el comportamiento computacional.** El SCP relaja a entero exacto ($LP = IP = 42$) y se cierra en un solo nodo; el BPP tiene una relajación débil (14.34 frente a 15) y depende del branch-and-bound. La misma teoría que se vio en clase se observó, medible, en los logs del solver.
 3. **La ruptura de simetría $y_j \geq y_{j+1}$ fue decisiva** en el Bin Packing: sin ella, el solver exploraría las $15!$ permutaciones equivalentes de etiquetas de contenedores. Una desigualdad de modelado superó a la fuerza bruta.
 4. **El análisis de sensibilidad reveló dos capas.** La estructural (contenedores $\propto 1/C$, reducción del 52 % entre $C=100$ y $C=200$, con mesetas) y la computacional (picos de tiempo no monótonos en capacidades de frontera, hasta $75 \times$ la mediana). La dificultad del BPP vive en la aritmética pesos-capacidad, no en el tamaño.
 5. **La interfaz importa.** El dashboard HTML autocontenido convirtió listas de índices en patrones legibles —barras de contenedores, matriz de cobertura, curva de sensibilidad— y mostró que, aun con la licencia gratuita de Gurobi, una capa visual delgada vuelve comunicable un resultado de optimización.
 6. **Limitación honesta y su mitigación.** La licencia restringida (2000 variables) impidió correr el BPP completo localmente; se documentó el interruptor `ITEM_LIMIT/WLS` y se acotó el experimento a 40 ítems sin perder la validez metodológica. La cota teórica para 120 ítems (47, frente al óptimo conocido 48) queda anotada para verificación al desbloquear la licencia.

Cierre. El proyecto confirma, con dos problemas clásicos y un solver industrial, que en programación entera el modelado (formulación fuerte, simetría) pesa tanto como el algoritmo, y que reportar *cuánto cuesta* una solución es tan informativo como la solución misma.